

Inheritance

Unlike many Object-relational mapping tools, OrientDB does not split documents between different classes. Each document resides in one or a number of clusters associated with its specific class. When you execute a query against a class that has subclasses, OrientDB searches the clusters of the target class and all subclasses.

Declaring Inheritance in Schema

In developing your application, bear in mind that OrientDB needs to know the class inheritance relationship. This is an abstract concept that applies to both [POJO's](#) and [Documents](#).

For example,

```
OClass account = database.getMetadata().getSchema().createClass("Account");
OClass company =
database.getMetadata().getSchema().createClass("Company").setSuperClass(account);
```

Using Polymorphic Queries

By default, OrientDB treats all queries as polymorphic. Using the example above, you can run the following query from the console:

orientdb>

```
SELECT FROM Account WHERE name.toUpperCase() = 'GOOGLE'
```

This query returns all instances of the classes `Account` and `Company` that have a property name that matches `Google`.

How Inheritance Works

Consider an example, where you have three classes, listed here with the cluster identifier in the parentheses.

```
Account(10) <|--- Company (13) <|--- OrientTechnologiesGroup (27)
```

By default, OrientDB creates a separate cluster for each class. It indicates this cluster by the `defaultClusterId` property in the class `OClass` and indicates the cluster used by default when not specified. However, the class `OClass` has a property `clusterIds`, (as `int[]`), that contains all the clusters able to contain the records of that class.

`clusterIds` and `defaultClusterId` are the same by default.

When you execute a query against a class, OrientDB limits the result-sets to only the records of the clusters contained in the `clusterIds` property. For example,

```
orientdb>
```

```
SELECT FROM Account WHERE name.toUpperCase() = 'GOOGLE'
```

This query returns all the records with the name property set to `GOOGLE` from all three classes, given that the base class `Account` was specified. For the class `Account`, OrientDB searches inside the clusters `10`, `13` and `27`, following the inheritance specified in the schema.